Sparse Dynamical Modeling of Baseball Pitching Kinematics from Vision-Derived Pose

Alexander C. Yu, M.S. Student

***Abstract*—Can sparse dynamical models learned from broadcast video describe baseball pitching motion? Ten kinematic features are extracted from 140 MLB pitching clips with MediaPipe pose and fitted by sparse identification of nonlinear dynamics (SINDy), and every score is compared against a no-model control for its own task. Three results follow. First, free-running reconstruction fails: no configuration beats a predictor that holds the initial pose fixed, and accuracy falls as model complexity rises. Second, diagnostic tests point to the state representation, rather than the estimator, as the primary limitation. Third, adding recent history to the state supports one-step prediction, but most of the resulting 0.39 skill over persistence is supplied by the centred smoothing filter; on unsmoothed pose it is 0.05 to 0.09. What holds under every smoother is a small lead, 0.01 to 0.03, over a fitted per-joint momentum model. Five intermediate results that looked positive did not survive their controls, and no biomechanical claim is made.**

***Index Terms*—sparse identification of nonlinear dynamics, SINDy, human pose estimation, sports biomechanics, model evaluation, baseline controls, time-series forecasting**

I. Introduction

Baseball pitching is a high-speed, multi-segment movement involving coordinated lower-body drive, trunk rotation, shoulder and elbow motion, stride mechanics, and rapid distal segment acceleration. Traditional biomechanical analysis often relies on laboratory motion capture, which produces high-quality trajectories but is expensive and difficult to scale; the canonical kinetic descriptions of pitching mechanics were established this way [1], and the sequence they describe — stride, pelvis rotation, upper-torso rotation, elbow extension, shoulder internal rotation, wrist flexion — is the proximal-to-distal kinetic chain this paper tests against in Section VII-D [2]. Broadcast or locally segmented video is more accessible, but vision-derived pose introduces noise, camera-view bias, missing landmarks, and two-dimensional projection limits. Markerless pose estimation has been validated against marker-based capture for gait, where joint-angle agreement is good for large sagittal motions and degrades for smaller out-of-plane rotations [3]; broadcast footage is a substantially harder case than the controlled multi-camera setting used in that comparison.

A. C. Yu is with the Department of Electrical and Computer Engineering, Rutgers University, New Brunswick, NJ 08901 USA (e-mail: acy29@scarletmail.rutgers.edu).

This project investigates whether a machine-vision and sparse-dynamics workflow can produce reproducible mathematical artifacts from baseball pitching motion. The goal is not to claim that the current dataset explains elite pitching mechanics. The goal is to establish a defensible technical pipeline that can ingest pose trajectories, condition them into stable feature matrices, estimate derivatives, and fit sparse dynamical models that can be compared across preprocessing and modeling configurations under a common evaluation protocol.

The contribution is threefold: a reproducible pipeline from broadcast video to sparse equations; an evaluation protocol that pairs every score with the no-model control for its own task and reports two aggregations; and a characterized boundary on what this data supports. Section IV states the protocol, Sections V through VII report the results, Section VIII records what the protocol caught, and Sections IX and X discuss limitations and conclude.

A. Related work

Sparse identification of nonlinear dynamics [4] has been extended in directions that matter here: the weak formulation for noise robustness [5], [6], bootstrap aggregation for the low-data high-noise regime [7], and relaxed-regularization optimizers [8]. All are available in PySINDy [9], [10] and all are evaluated in Section V.

Closest to this work in application is Rosenberg et al. [11], which fits sparse hybrid dynamics to centre-of-mass trajectories during walking and reports individual-specific, interpretable models. Two differences shape what is achievable. That work uses laboratory motion capture, where the measurement noise is far below the signal, and it models a periodic gait cycle, where a low-dimensional template is a good description. Pitching is a single ballistic transient recorded here from broadcast video, and Sections V and VI show both differences bite.

For the pose layer, markerless estimation has been validated against marker-based capture for gait, with good agreement on large sagittal motions and degradation on smaller out-of-plane rotations [3]; broadcast footage is a harder case again. On this dataset specifically, Piergiovanni and Ryoo [12] predict pitch speed and type directly from video and describe the task as extremely difficult, which is the prior against which the label study of Section VII-D should be read. Laboratory studies do identify kinematic correlates of ball velocity [13] and of trunk-rotation timing [14], so

the question this paper can address is not whether such correlates exist but whether they survive this medium.

The delay-coordinate results in Sections VI and VII rest on Takens' embedding theorem [15] and the practice of obtaining linear models of nonlinear dynamics from delay coordinates [16], with dynamic mode decomposition [17] as the spectral counterpart used in the Section VII-D label study.

B. Research questions

Two questions are primary, and they differ only in the task being scored:

1. Do the fitted sparse systems outperform a no-dynamics control baseline on held-out free-running reconstruction?
2. Do they outperform the corresponding controls on short-horizon prediction — persistence, and the simplest fitted model of recent motion — and does the answer differ from question 1?

One diagnostic question follows from the answers:

3. Where the two answers differ, is the limiting factor the model class, the state representation, the clip heterogeneity, or the pose noise floor?

Three further questions are engineering prerequisites rather than findings, and are answered in passing: whether raw or vision-derived skeletal trajectories can be converted into standardized pitching feature matrices without manual per-clip intervention (Section III), whether derivative and sparse coefficient matrices regenerate reproducibly from those feature matrices (Section XI), and whether the fitted systems are numerically stable on held-out clips (Section V-A — they are, and Section IV explains why that is close to uninformative).

II. Data

A. Stage A validation data

Stage A uses UTD-MHAD [18] as a proxy human-motion dataset. The selected proxy actions are labeled as `batting_like` and `pitching_like` to test rotational and ballistic motion processing. These labels are methodological categories and do not imply that UTD-MHAD motions reproduce baseball mechanics.

B. Stage B baseball data

Stage B uses local baseball clips drawn from the MLB-YouTube dataset [12], represented in `data/baseball/metadata/clip_manifest.csv`, with local-media split files under `data/baseball/splits/` and standardized pose payloads registered under `data/baseball/standardized_pose/`. MediaPipe pose exports are imported into a 15-joint schema compatible with the Stage A skeleton format.

The current pitching pool is **140 clips** (`data/baseball/splits/pitching_local_pool.csv`), split into **111 training clips** (`pitching_local_train.csv`) and **29 held-out test clips** (`pitching_local_test.csv`). All clips come from a single consistent camera view (`broadcast_centerfield`). The effective sampling interval is `dt = 0.016683 s` (approximately 59.9 fps).

Each clip additionally carries `pitch_type` and `pitch_speed_mph` labels in the pool CSV. These labels are not used by the dynamics pipeline described here; they are recorded for the label-validated representation study scoped in `docs/superpowers/specs/2026-06-23-representation-study-design.md`. Piergiovanni and Ryoo [12] attempt pitch-speed and pitch-type prediction from the same broadcast footage and describe it as extremely difficult, which is the relevant prior for that study.

Generated arrays, processed features, models, and result summaries are intentionally excluded from version control by `.gitignore`. They are reproducible from the commands in Section XI.

III. Pipeline

The workflow has two stages. Stage A validates the general motion-processing and sparse-modeling infrastructure on UTD-MHAD — data loading, skeleton standardization, smoothing, derivative estimation, model fitting, coefficient export — so its artifacts are infrastructure checks rather than baseball findings. Stage B replaces only the input-pose layer with baseball-specific standardized pose and keeps everything downstream, which keeps video and pose-estimation issues separate from the modeling workflow. The pipeline is described here only as far as the results require. The code is public, and Section XI says how every number is regenerated.

A. Pose standardization

Each baseball pose clip is stored as a compressed `.npz` payload containing:

- `coordinates`: pose array with shape `(time, joints, coordinate_dims)`
- `joint_names`: ordered joint labels
- `metadata`: JSON-serializable clip and pose-source metadata

The preferred joint schema contains 15 joints: head, neck, spine, shoulders, elbows, wrists, hips, knees, and ankles. The current adapter maps the larger MediaPipe landmark set into this schema. Pose is produced by the MediaPipe framework [19] using its BlazePose topology [20], which is a monocular on-device estimator rather than a calibrated multi-camera system.

B. Preprocessing

The baseball preprocessing script repairs leading and trailing missing values by edge fill, interpolates internal gaps, centers trajectories at the hip-center proxy, optionally applies orientation and scale normalization, smooths

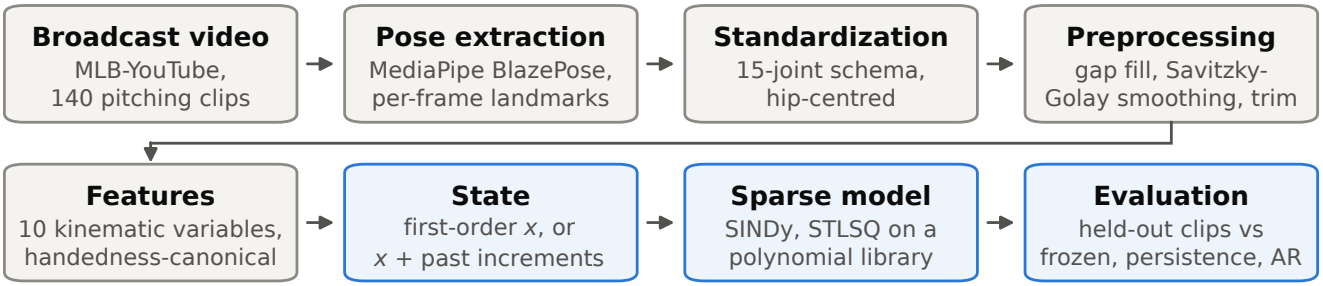


Fig. 1. The pipeline, read left to right and then along the second row. The grey boxes (top row and Features) turn broadcast video into a 10-variable kinematic time series, and the blue boxes are the modeling. Two places carry the paper's argument. The State box is where the first-order state x is extended with past increments (Sections VI and VII). The Evaluation box scores every model against a no-model control matched to its task (Section IV).

coordinates with a Savitzky-Golay filter [21], and trims low-motion frames around the action window.

The operative baseball configuration is `config/preprocessing.baseball.unaligned.handedness.yaml`, which uses:

- center joint: `hip_center`
- orientation normalization: disabled
- scale normalization: disabled
- internal gap interpolation: enabled, maximum gap 30 frames
- coordinate smoothing: Savitzky-Golay, window length 11, polynomial order 3
- motion trimming: enabled, energy threshold ratio 0.2, 4 padding frames, minimum 25 frames

C. Feature extraction

The current Stage B pitching state vector has 10 variables:

1. `shoulder_rotation_angle`
2. `hip_rotation_angle`
3. `hip_shoulder_separation_angle`
4. `dominant_elbow_flexion_angle`
5. `trail_elbow_flexion_angle`
6. `dominant_knee_flexion_angle`
7. `trail_knee_flexion_angle`
8. `lead_knee_lift`
9. `stride_ankle_separation`
10. `dominant_wrist_speed`

The state is chosen to cover the segments the throwing sequence runs through [2]: stride via `stride_ankle_separation` and `lead_knee_lift`, pelvis and trunk via the two rotation angles and their difference, and the arm via elbow flexion and wrist speed. `hip_shoulder_separation_angle` is included because pelvis-to-trunk separation and the timing of trunk rotation are the parameters laboratory work associates with ball velocity and with shoulder loading [13], [14].

Features are canonicalized by handedness: the configuration sets `default_handedness: R` and `infer_handedness_for_pitching: true`, so dominant/trail designations are resolved per clip rather than fixed to a body side. Angles are unwrapped (`angle_unwrap: true`).

Event alignment is **disabled** in the operative configuration (`features.alignment.enabled: false`). An earlier configuration track aligned pitching clips around stride-ankle separation with lead-knee lift as a secondary signal; that track is retained as `config/preprocessing.baseball.aligned.yaml` but is not the operative default. It was last compared on the superseded whole-state score and has not been re-scored (Section IX).

Derivatives are estimated with Savitzky-Golay differentiation using window length 9 and polynomial order 3.

D. Sparse model identification

Sparse models are fit by the SINDy method of Brunton et al. [4], which selects a parsimonious set of library terms for `dx/dt` by sequentially thresholded least squares (STLSQ). The implementation is PySINDy [9], [10], using a polynomial library with bias and interaction terms, feature scaling enabled, and STLSQ sparse regression. The operative configuration uses:

- polynomial degree: 1
- STLSQ alpha: 0.001
- maximum STLSQ iterations: 20
- model-selection metric: sparsity-weighted R2 (`selection_metric: sparsity_weighted_r2`)
- complexity penalty: 5e-05 per term
- validation fraction for selection: 0.22 (24 of 111 training clips), seed 7

The STLSQ threshold is **not fixed**; it is swept over `{0.1, 0.15, 0.2, 0.3, 0.5}` and selected by the sparsity-weighted score on the held-out selection split. For the operative first-order configuration the sweep selects **threshold 0.2**, yielding **26 equation terms**. Section VIII documents a degeneracy in this selection rule.

A weak-form variant is available at `config/preprocessing.baseball.unaligned.handedness.weakform.yaml`, gated by `modeling.weak_form.enabled`. This uses the weak, integral formulation of Messenger

and Bortz [5], [6], which replaces pointwise derivative estimates with integrals against test functions and is therefore substantially more robust to measurement noise — the motivation for trying it on broadcast-derived pose. It fits a `WeakPDELibrary` SINDy over stacked per-trajectory weak systems across all 111 training clips, then transplants the resulting coefficients into a plain polynomial-library SINDy so the standard predict and simulate paths apply unchanged. Because `WeakPDELibrary` places its test-function subdomains via the global NumPy RNG with no seed parameter, a single draw is neither reproducible nor low-variance; the implementation seeds locally (restoring prior RNG state) and stacks `n_seeds` independent subdomain draws. The operative weak-form settings are `K: 100`, `seed: 7`, `n_seeds: 5`.

STLSQ is one sparse optimizer among several; relaxed-regularization alternatives are surveyed and unified by Champion et al. [8], and Section VI reports what changing the optimizer does here. An ensemble variant at `config/preprocessing.baseball.unaligned.handedness.ensemble.yaml` wraps the optimizer in bootstrap aggregation following Ensemble-SINDy [7], which is intended to improve robustness in the low-data, high-noise regime that applies here.

The model outputs include equations, coefficient matrices, model-selection logs, scaler metadata, per-trial metrics, and summary CSVs.

IV. Evaluation protocol

This section is placed before the results because the protocol, not any single number, is what makes them interpretable. Five of this project's intermediate findings were false positives that only a control could catch, and Section VIII documents them. Held-out evaluation computes one-step derivative-prediction metrics and free-running reconstruction metrics. Reconstruction integrates the fitted system forward from the first observed state for `reconstruction_frames: 40` frames using the **RK4** integrator (`evaluation.integrator: rk4`; an `euler` option is retained and was marginally worse, core R2 -29.99 vs -29.56 on the first-order baseline).

Two reconstruction metrics are reported:

- **Whole-state reconstruction R2** — variance-weighted across all 10 state columns. This metric is *not* comparable across different state-vector definitions, because high-variance columns dominate it. For the 10-D first-order state, `dominant_wrist_speed` inflates the score.
- **Core-position reconstruction R2 / NRMSE** — a state-comparable metric computed only over the 8 shared `CORE_POSITIONS` columns (all state variables except `hip_shoulder_separation_angle` and `dominant_wrist_speed`), using `r2_score(..., multioutput="uniform_average")` so no single column dominates. NRMSE normalizes each column's RMSE by that column's standard deviation, making it unit-free.

Core-position R2 is additionally reported at rollout horizons `{5, 10, 20, 40}` frames from the same forward simulation.

Control baseline. Reconstruction R2 is measured against the per-window mean, which is a harsh reference when the core positions move little over a short rollout. To make the reconstruction numbers interpretable, `src/preprocessing/evaluate_static_baseline.py` scores a **frozen model** — zero derivative, so the state is held at its first observed value for the whole rollout — under the identical metric, horizons, and test split. This encodes no dynamics whatsoever and therefore establishes the floor that any fitted model must clear for its reconstruction score to carry information.

Simulation stability (finite and bounded trajectories) is reported separately. Stability is a necessary but very weak condition and is not evidence of predictive accuracy: a model whose coefficients are near zero is bounded by construction.

Short-horizon prediction and skill over persistence. Free-running reconstruction and one-step prediction are different tasks with non-comparable scores, and each needs its own control. For one-step prediction the correct control is not the frozen model but **persistence** — predict `x(t) = x(t-1)`. At approximately 60 fps consecutive frames are nearly identical, so persistence is a very strong baseline: it alone scores R2 = 0.968 on the state. State R2 is therefore saturated by the near-identity component and cannot separate models. The reported metric for this task is instead **skill over persistence** on the increment:

```
skill = 1 - SSE(model) / SSE(zero-increment)
```

where the target is `delta(t) = x(t+1) - x(t)`. Expressing accuracy against a naive benchmark rather than in absolute units is standard practice in forecast evaluation [22]. This is the standard mean-square-error skill score, in which accuracy is expressed relative to a trivial reference forecast rather than in absolute terms [23]; persistence is the reference here, and the climatological analogue — the window mean — is what the reconstruction R2 above uses. Skill is 0 for persistence by construction, and positive skill means predictive structure beyond "nothing changes."

Local motion controls. Persistence is the no-model control, but it is not sufficient for a model that is given history. A model that sees the last few frames should also clear the simplest fitted model that uses them: per-joint autoregression on each joint's own past increments, whose one-coefficient case is damped momentum. The rule is that a fitted model's control is fitted too, on the same rows. Unfitted least-squares polynomial extrapolation over recent frames is reported alongside, as the parameter-free version of the same idea.

`evaluate_baseball_sindy.py` computes this on every run, as `persistence_skill_pooled` and `persistence_skill_uniform` per clip and `mean_persistence_skill_*` in the summary, so the metric the project judges models by is emitted by the pipeline itself rather than by a separate diagnostic. The reconstruction metrics remain alongside it.

Aggregation is reported two ways, and disagreement is treated as a null result. Core positions span roughly a 13x range in per-frame signal scale (`lead_knee_lift` 0.012 versus `dominant_elbow_flexion_angle` 0.158, about 160x in variance), so pooled sum-of-squares is decided almost entirely by the largest-amplitude columns while uniform per-column averaging weights all eight equally. Every skill number in Section V is reported under both. Where the two disagree in sign, this paper reports no win rather than selecting the favorable aggregation. `core_position_metrics` uses uniform averaging, so uniform is the figure comparable to the rest of the project.

V. Free-running reconstruction fails

All results below were produced on 2026-08-12 against the same 140-clip pool (111 train / 29 test), RK4 integrator, and 40-frame reconstruction. Every number is backed by a CSV under `results/`; the consolidated table is `results/evaluation/baseball/model_comparison_2026-08-12.csv`.

A. The operative configuration

The operative first-order configuration is fitted on the 111 training clips. Its threshold sweep selects STLSQ threshold 0.2 and 26 terms, with a global training derivative R2 of 0.0063 (`results/equations/baseball/train/summary.csv`). On the 29 held-out clips the mean derivative R2 is -0.0184. Skill over persistence is +0.0014 pooled and -0.0041 uniform, so the model has no short-horizon predictive structure beyond persistence. Section VI points to the state rather than the estimator, and Section VII recovers the structure by adding history. Every simulation is stable, but core-position reconstruction R2 is -29.56 (`results/evaluation/baseball/train_on_test/summary.csv`).

B. Every model against the frozen control

`core_position_recon_r2` is the state-comparable metric; higher (less negative) is better. Because every configuration was scored on the same 29 held-out clips, the comparison with the control is **paired**. The interval column bootstraps [24] the per-clip difference over clips, not frames, since frames within a clip are strongly dependent (`src/analysis/paired_control_intervals.py`).

On the whole-state score, which Section IV shows is not comparable across configurations, the ordering differs: the operative model scores -2.80 and weak-form -6.72.

Both intervals exclude zero, so the headline claim survives: these models are worse than doing nothing, not merely indistinguishable from it.

The means hide one detail. Weak-form beats the control on **15 of 29 clips**, a bare majority, yet its average is worse, because its losses are far larger than its wins. The aggregate conclusion is driven by tail behaviour on a minority of clips rather than by uniform inferiority.

Three observations follow directly:

1. **No fitted model beats the frozen control at any horizon.** The best fitted model (weak-form, core R2 -7.89) is outscored by a predictor that holds the initial pose constant (-5.57). Under this metric, every dynamical model tried is worse than assuming the pitcher does not move.
2. **The ordering is monotonic in model complexity, in the wrong direction.** Core R2 degrades as term count rises ($0 \to 1 \to 26 \to 40 \to 101 \to 131$ terms). Added expressivity strictly hurts held-out reconstruction, which is the signature of fitting noise rather than dynamics.
3. **Stability is uninformative here.** Every configuration reports a stable simulation fraction of 1.0, including models that are numerically bounded only because their coefficients are near zero.

C. Rollout horizon

The delay-augmented increment model of Section VII, rolled out autoregressively against persistence (`results/analysis/increment_skill/part_b_horizon_crossover.csv`):

Teacher forcing confirms that the rollout is implemented correctly: supplying the true history at each step gives +0.7569 core R2, against +0.8453 for persistence (`results/evaluation/baseball/forecast_baselines/summary.csv`). The free-running loss is therefore error compounding, the growth with horizon that follows whenever a model is fed its own predictions [25].

The pooled-uniform disagreement comes from amplitude spread across the core columns. Pooled sum of squares is decided by the large-amplitude angles, on which the model helps. The small-amplitude columns, on which it does badly, count equally under uniform averaging. The H = 240 row, core R2 -1.87 against persistence -2.73, is the only sign of the crossover one would expect as persistence degrades. Uniform skill is still negative there and only 13 clips are long enough to qualify, so it is a direction to test with longer clips, not a result.

The frozen model also beats every other free-running control under a common seed and scoring window. It scores -5.56, against -42.08 for the rolled-out `delay_linear` model, -45.01 for a phase-warped template of the mean training pitch, and -1096.28 for constant-velocity extrapolation. The template result is worth noting on its own: at this pose quality, pitching is not stereotyped enough for template matching either.

TABLE I

HELD-OUT 40-FRAME CORE-POSITION RECONSTRUCTION R2 AGAINST THE FROZEN (ZERO-DYNAMICS) CONTROL, 29 TEST CLIPS. READ THE CORE R2 COLUMN FROM TOP TO BOTTOM: IT FALLS MONOTONICALLY AS THE TERM COUNT RISES. THE PAIRED DIFFERENCE IS A BOOTSTRAP OVER CLIPS (10,000 RESAMPLES), AND BOTH COMPUTED INTERVALS EXCLUDE ZERO. DASHES MARK ROWS WITHOUT INTERVALS, WHICH WOULD NEED REFITTING PER RESAMPLE. WEAK-FORM IS THE FIRST-ORDER MODEL FITTED WITH THE MULTI-SEED WEAK FORMULATION.

model	terms	core R2	paired difference from control [95% CI]	clips beating control
frozen control	**0**	**-5.5707**	—	—
weak-form	1	-7.8922	-2.32 [-4.98, -0.27]	15/29
first-order (operative)	26	-29.5553	-23.98 [-48.21, -4.22]	11/29
first-order ensemble	40	-36.9650	—	—
second-order	101	-2269.04	—	—
second-order ensemble	131	-2428.87	—	—

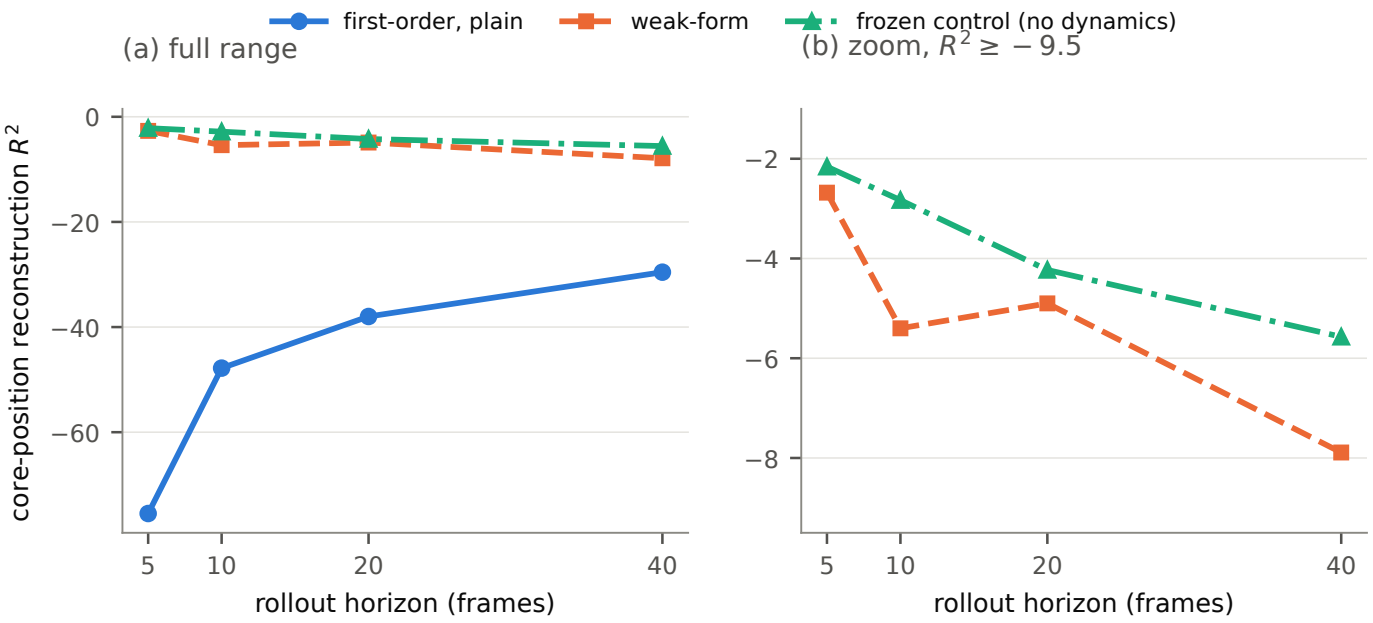


Fig. 2. Core-position reconstruction R2 against rollout horizon for the plain first-order model, the weak-form model and the frozen control. (a) Full range: the plain model lies far below the other two at every horizon, though it improves as the horizon grows. (b) The same data restricted to R2 above -9.5. Note that the frozen-control line (dash-dot) is above the weak-form line at every horizon. The gap is smallest at 5 frames (-2.16 against -2.68) and widest at 40 frames (-5.57 against -7.89).

TABLE II

ROLLED-OUT INCREMENT MODEL AGAINST PERSISTENCE BY HORIZON. POOLED AND UNIFORM SKILL DISAGREE IN SIGN AT EVERY HORIZON, SO NO MULTI-STEP WIN IS CLAIMED.

H (frames)	H (sec)	clips	skill (pooled)	skill (uniform)	core R2 model	core R2 persistence
5	0.083	29	+0.1751	-68.15	-354.00	-8.38
10	0.167	29	+0.1906	-23.47	-110.86	-6.76
20	0.334	29	+0.2145	-24.82	-68.56	-5.92
40	0.667	29	+0.1269	-17.19	-43.36	-5.63
80	1.335	28	+0.1385	-7.72	-33.60	-5.14
160	2.669	21	+0.1517	-4.13	-9.30	-3.16
240	4.004	13	+0.1438	-0.4955	-1.87	-2.73

VI. DIAGNOSING THE FAILURE

Four candidate explanations for the failure are tested in turn. None accounts for it, and the evidence points to the state representation as the primary limitation.

A. *Phase and clip heterogeneity*

Phase and clip heterogeneity are tested with plain ridge regression rather than STLSQ so that sparse-selection artifacts cannot confound the comparison, with GroupKFold over clips so no clip appears in both train and test, since frames inside a clip are strongly dependent and a naive fold split over a time series is optimistic [26] (`src/preprocessing/diagnose_derivative_fit.py`, `results/analysis/derivative_fit_diagnostics.csv`):

Phase does not help. Adding normalized phase φ — as an additive term, as a modulating interaction A(φ)x, and as piecewise-in-phase bins — produces no gain in any of

TABLE III
HELD-OUT DERIVATIVE R2 BY STATE REPRESENTATION, PLAIN RIDGE WITH GROUPKFOLD OVER CLIPS. PHASE ADDS NOTHING; HALF OF THE DELAY-EMBEDDING GAIN IS RECOVERED NUMERICAL DIFFERENTIATION.

representation	held-out derivative R2	reading
`global_linear` — [1, x]	-0.0004	structural match to the operative degree-1 SINDy
`global_linear_phase` — [1, x, φ]	-0.0003	no effect
`global_phase_interact` — A(φ)x	+0.0071	no meaningful gain
`phase_binned_k4` / `k8`	+0.0019 / -0.0008	no gain
`delay_embed` τ=1, d=3	**+0.6901**	large apparent gain
`finite_diff_control` (unfitted backward difference)	+0.3507	**half the gain is rediscovered differentiation**
`per_clip_within` (fit early 70%, predict late 30%)	-38.89	within-clip temporal extrapolation fails
`per_clip_oracle` (in-sample)	+0.2500	upper bound for a per-clip model

the three forms tried, so a phase-dependent map is unlikely to explain the failure of `dx/dt = f(x)`.

Clip heterogeneity is not the main limit; the state representation is the more likely one. The per-clip oracle scores only +0.25 *in sample*, lower than a delay-embedded model scores on *held-out* clips (+0.69). A single clip cannot be fit well in the plain 10-D state even when the model is allowed to see it. This argues against per-pitcher, per-pitch-type and per-phase conditioning, which all address heterogeneity.

The delay-embedded rows follow Takens [15], whose embedding theorem motivates reconstructing a state from lagged observations of a single trajectory, and the specific use of delay coordinates to obtain a linear model of nonlinear dynamics follows the HAVOK construction of Brunton et al. [16]. Their apparent gain must nonetheless be discounted: an unfitted backward difference already reaches +0.3507, so roughly half of the +0.69 is numerical differentiation recovered from position history rather than learned dynamics.

B. The pose noise floor

Across all 140 clips, per-feature signal-to-noise is 2.2 to 4.4. The ratio is per-frame signal over the high-frequency residual. It is highest for `lead_knee_lift` and lowest for `dominant_wrist_speed` and the elbow angles (`results/analysis/increment_skill/part_c_noise_floor.csv`). Increment lag-1 autocorrelation is 0.35 to 0.88, so increments are far from white and carry predictable structure. **The noise floor does not appear to be the limiting factor**, and short-horizon prediction is not capped by pose quality at this level. Higher-quality pose remains desirable for biomechanical interpretation, but it is not a precondition for the Section VII result. These autocorrelations are measured on centred-smoothed features and are partly imposed by the filter; Section VII-C measures how much.

The residual estimate is an upper bound on achievable precision, not a measurement of raw MediaPipe jitter. The features are already Savitzky-Golay smoothed, so this measures leftover high-frequency content in the modeled features. It is a floor on the floor.

C. The sparse estimator

Section VII establishes short-horizon structure using a delay-embedded ridge model, which is not what this pipeline produces. Whether the fitted STLSQ equations capture that structure determines how broadly the result can be claimed. `src/preprocessing/evaluate_sindy_increment_skill.py` scores the fitted models on that task. `src/analysis/diagnose_sparse_fit_loss.py` holds the design matrix and target fixed and varies only the estimator, over 20 clip-level resplits (`results/analysis/sparse_fit_loss.csv`).

The operative equations capture none of the structure, but sparsification is not why. The operative first-order model has no short-horizon skill. Dense ridge and the best STLSQ differ by 0.0002 on the first-order state and by 0.0000 on the second-order state, and their intervals overlap almost completely. There is a real but small threshold cost. At the operative threshold 0.2 the first-order fit is zeroed entirely: +0.0005 against +0.0128 at threshold 0.01, which is why the fitted model scores zero. But the amount at stake is 0.013.

The Savitzky-Golay derivative reference (0.70 skill on the first-order targets) is not a ceiling on what these models could reach. The derivative is estimated over a window that includes future frames, so scoring it against the forward increment measures already knowing the answer. It is an oracle, not a bound. The same input-copying problem appears in the second-order model's 0.8905 skill on *position* increments. Velocity is an input column of that state, so `d(position)/dt` is read off the input rather than learned, and the model ties its own reference to within 0.0003. Only the acceleration row above is a real task for that state. There the second-order model reaches 0.0826, from the configuration that fails catastrophically on reconstruction (core R2 -2269, Section V).

The state's information content is the main limit. A dense linear model of position alone reaches only +0.0099. Position does not determine the next increment, so the sparse fit had almost nothing to lose. This agrees with Section VI-A, where plain ridge on `[1, x]` scored -0.0004 on the derivative. Recent history is what predicts the increment. On the identical target and the identical 29 test clips, adding two frames of history to the same ridge

TABLE IV
UNIFORM SKILL OVER PERSISTENCE BY ESTIMATOR, WITH 95% INTERVALS OVER RESPLITS. COMPARE THE LAST TWO COLUMNS: DENSE AND SPARSE AGREE TO WITHIN 0.0002 ON BOTH STATES, SO SPARSIFICATION COSTS NOTHING DETECTABLE. THE FIRST COLUMN IS THE PIPELINE'S OWN FITTED EQUATIONS ON THE SINGLE HELD-OUT SPLIT.

state	target	fitted SINDy, held-out split	dense RidgeCV, 20 resplits	best STLSQ, 20 resplits
first-order	position increment	-0.0010	**+0.0099** [+0.0086, +0.0112]	+0.0097 [+0.0083, +0.0111]
second-order	rate increment (acceleration)	+0.0826	**+0.1311** [+0.1181, +0.1440]	+0.1310 [+0.1181, +0.1440]

model raises uniform skill from +0.0099 to **+0.3874**. An autonomous first-order SINDy model has no mechanism to use that history.

Two consequences follow for what this paper may claim. First, **the Section VII positive result does not belong to the operative SINDy configuration**. It belongs to the data and to a delay-augmented state. Second, the loss sits in the *state representation* rather than the sparse fit. Over joint positions, `dx/dt = f(x)` does not capture what predicts the next increment, and no sparse estimator, threshold or selection rule tried here changes that.

VII. SHORT-HORIZON PREDICTION SUCCEEDS

A delay-embedded ridge model predicts the one-step increment on the 29 held-out clips with skill over persistence of **+0.3874 uniform and +0.2816 pooled** (`src/preprocessing/diagnose_increment_skill.py`). **This is the project's only positive result that survives its control, and it is positive under both aggregations.** The model captures about 39 percent of held-out increment variance beyond persistence.

State R2 cannot separate the models: 0.9777 for the model against 0.9680 for persistence. A 39 percent improvement in the predicted dynamics appears as a third-decimal difference on the state, because the near-identity component dominates the state metric. Skill over persistence on the increment is the metric that does separate them.

Predicting the next state instead of the increment gives the same skill (0.2815 against 0.2816 pooled). `RidgeCV` selects alpha of order 1e-4, so the two targets span the same linear space and the choice of target does not matter.

A. Sparse identification on a delay-augmented state

Section VI localizes the failure to the state rather than the estimator. This section tests the implied fix: fit the project's own sparse estimator, STLSQ, on a delay-augmented state and score it against persistence. Design is `[x(t), delta(t-1), ..., delta(t-d+1)]`, target is the next-frame increment of the eight core positions, and history depth is swept rather than assumed (`src/analysis/fit_delay_sindy.py`, `results/analysis/delay_sindy/`).

With no history (depth 1) the sparse fit has 8 terms and scores +0.0109 uniform. One frame of history (depth 2, 21 terms) raises that to +0.3739. Skill saturates at depth 3: +0.3881 with 30 terms, then +0.3884 at depth 4 and +0.3858 at depth 5. Pooled skill has the same shape: +0.0155, +0.2685 and +0.2817 at depths 1 to 3.

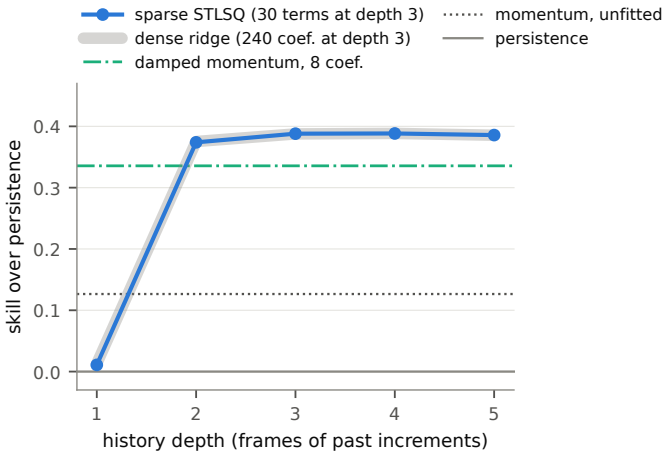


Fig. 3. Uniform skill over persistence on held-out increments against history depth. Note three things. Almost the whole gain arrives between depth 1 and depth 2. The curve is flat from depth 3. The sparse line stays inside the pale dense-ridge band at every depth; at depth 3, 30 sparse terms match 240 dense coefficients. The horizontal lines are the controls: the unfitted 2-frame momentum (dotted) and 8-coefficient damped momentum (dash-dot). The sparse fit clears damped momentum only from depth 2 on.

Sparse identification retains essentially all of the available skill. At depth 3 a 30-term sparse model scores +0.3881 uniform, against +0.3877 for dense ridge with 240 coefficients. The sparse fit is not worse, and it is an order of magnitude smaller. This is the first result in this project that belongs to the pipeline's own method rather than to a baseline.

B. Against local motion

Every discovered equation is dominated by the target's own previous increment, so persistence is not the control that matters here. The fitted model uses history, and Section IV requires it to clear the simplest fitted model that also uses history: a joint continuing to move as it was moving. Two families of local motion model are scored on one shared row set (`control:` rows in `skill_by_depth.csv`). Least-squares polynomial extrapolation over the last W frames has no fitted parameters. Per-joint autoregression on that joint's own past increments is fitted on train, with no levels and no coupling; its one-lag case is damped momentum, `delta_hat_j(t) = a_j delta_j(t-1)`.

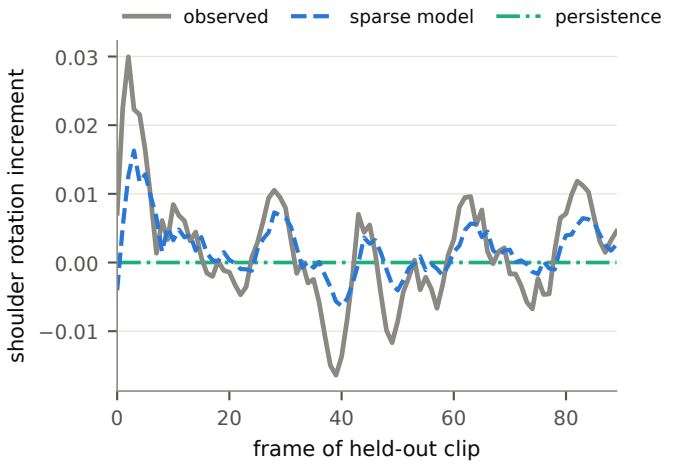


Fig. 4. Observed and predicted one-step shoulder-rotation increments over the first 90 frames of the longest held-out clip. Persistence predicts a zero increment throughout. Look at the timing: the sparse model follows the sign changes of the observed increment about one frame late. It also underestimates the largest excursions, near frames 2 and 38. The increment is plotted rather than the state because persistence alone scores 0.968 on the state, so state-space curves overlap.

The resplit margins use a stabilized 27-column design. `hip_shoulder_separation_angle` is shoulder rotation minus hip rotation, and the identity holds near-exactly on some clip subsets. With it in the design, the unrestricted fit diverged on 2 of 40 resplits: condition number 3.0e6 and cancelling coefficients of ±6773. Dropping the derived feature removes the divergence at a cost of 0.0009 skill (`src/analysis/stabilize_delay_fit.py`). A larger STLSQ ridge penalty does not help, which points to a true rank deficiency rather than ordinary ill-conditioning.

Polynomial extrapolation fails, and fails worse as the window widens. A wider fit extrapolates the average velocity over the window, which lags an accelerating motion, and a quadratic overshoots. Neither is a usable model of the next frame.

Damped momentum works, and it accounts for most of the skill. Each increment is predicted as a fixed fraction of the last, 0.40 to 0.80 depending on the joint. That reaches +0.3356 uniform with 8 coefficients, 87 percent of the sparse fit under the operative smoothing. Section VII-C shows the share falls to about half without smoothing. This is Section VI's diagnosis in its plainest form: the first-order state did not lack a richer function of position. It lacked the joint's own recent motion, and one coefficient per joint recovers most of it.

The sparse fit's margin is small, and it holds. On the held-out split the sparse fit leads per-joint AR(3) by +0.026 uniform and +0.033 pooled. Over 40 resplits the margins are +0.0187 uniform and +0.0237 pooled over AR(3), and +0.0403 and +0.0443 over damped momentum. All four are positive on all 40 resplits. About half of the margin over damped momentum comes from deeper own history, which AR(3) also has. The rest comes from levels and cross-joint terms, the coupling Section VII-D finds to be weak. The sparse fit's lead over local motion is therefore real but modest: roughly 0.02 skill, against the 0.34 that momentum alone provides.

C. How much of it is the smoothing filter

Every number above is computed on coordinates smoothed by a centred Savitzky-Golay filter, window 11, order 3 (Section III-B), so the smoothed `x(t)` already contains raw frames up to `t+5`, and `dominant_wrist_speed`, a design input, is a central difference that reads frame `t+1`. This is the mechanism that made the derivative reference an oracle in Section VI-C, acting more weakly on the state itself. `src/analysis/test_causal_smoothing.py` measures how much of the skill it supplies. Every clip is rebuilt from raw pose with the pipeline's own functions, changing only the smoother, and keeps the trim window and handedness the operative run recorded, so the comparison is paired clip for clip; the centred rebuild reproduces the stored features exactly. The causal variants also compute wrist speed by backward difference, so no input reads a future frame (`results/analysis/delay_sindy/causal_smoothing.csv`):

The level is set mostly by the filter. Without smoothing the sparse fit reaches +0.0510 uniform and +0.0866 pooled, against +0.3858 and +0.2781 under the centred filter. A causal filter is not a clean fix either: it imposes predictability too, in proportion to how hard it smooths, because the next smoothed value is mostly a weighted sum of frames already seen. The order-1 one-sided fit averages the most and scores +0.2801; the order-3 fit tracks the raw signal more closely and scores +0.0939. The skill of any smoothed series therefore measures the filter and the motion together, and only the unsmoothed row measures the motion alone. That row is small but positive under both aggregations, so short-horizon structure is present in the raw pose; its size is roughly an eighth of the operative figure uniform and a third pooled.

The lead over local motion is what survives. Under every smoother the sparse fit beats per-joint AR(3) by +0.013 to +0.028 uniform, positive on all 40 resplits under both aggregations, so the ranking in Section VII-B is a property of the data rather than of the preprocessing. The share that momentum explains is not: damped momentum reaches 87 percent of the sparse fit under the centred filter but about half of it on raw pose (+0.0268 of +0.0510 uniform). With the filter's contribution removed, what the sparse fit adds over each joint's own history is a larger part of a much smaller total.

D. The pipeline reproduces it

The results above come from an analysis script. To make them reproducible from the standard pipeline commands, the feature extractor takes a `features.delay_embedding` block (`enabled`, `depth`, `drop_features`), so a delay-augmented state is a first-class configuration

TABLE V

LOCAL MOTION MODELS AGAINST THE SPARSE FIT (DEPTH 3, THRESHOLD 0.05) ON THE SAME 6,070 HELD-OUT ROWS, FROM FRAME 8 ON SO EVERY WINDOW IS AVAILABLE; POLYNOMIAL WINDOWS ARE SELECTED ON TRAIN. READ DOWN THE UNIFORM COLUMN. EVERY PARAMETER-FREE EXTRAPOLATOR EXCEPT THE 2-FRAME ONE IS NEGATIVE. DAMPED MOMENTUM, WITH 8 COEFFICIENTS, REACHES 87 PERCENT OF THE SPARSE FIT. THE SPARSE FIT'S LEAD OVER AR(3) IS THE SMALLEST MARGIN IN THE TABLE. THE LAST COLUMN PAIRS ROWS WITHIN EACH RESPLIT, WITH A 95% INTERVAL. RESPLITS SHARE TRAINING CLIPS, SO THE INTERVALS ARE NARROWER THAN INDEPENDENT SAMPLES WOULD GIVE.

model	params	skill (pooled)	skill (uniform)	sparse-fit margin, uniform, 40 resplits
persistence, zero increment	0	0.0000	0.0000	—
local linear, 2 frames: `delta_hat(t) = delta(t-1)`	0	-0.0729	+0.1265	—
local linear, least squares over 3 to 9 frames	0	-0.28 to -2.41	-0.10 to -2.78	—
local quadratic, 3 frames: constant acceleration	0	-1.5447	-0.9875	—
local quadratic, least squares, W=4 (train-selected)	0	-1.0642	-0.6333	—
per-joint AR(1): damped momentum	8	+0.2242	+0.3356	+0.0403 [+0.0380, +0.0425]
per-joint AR(2)	16	+0.2420	+0.3558	—
per-joint AR(3)	24	+0.2449	+0.3595	**+0.0187** [+0.0171, +0.0204]
sparse STLSQ, all terms	31	**+0.2781**	**+0.3858**	—

TABLE VI

SKILL OVER PERSISTENCE, POOLED / UNIFORM, ON THE HELD-OUT SPLIT AND THE SECTION VII-B ROWS, BY COORDINATE SMOOTHER. THE LAST COLUMN IS THE PAIRED UNIFORM MARGIN WITH ITS 95% INTERVAL; IT IS POSITIVE ON 40 OF 40 RESPLITS UNDER BOTH AGGREGATIONS IN EVERY ROW.

coordinate smoothing	sparse fit	damped momentum	per-joint AR(3)	lead over AR(3), 40 resplits
centred, window 11, order 3 (operative)	+0.2781 / +0.3858	+0.2242 / +0.3356	+0.2449 / +0.3595	+0.0187 ± 0.0017
one-sided, window 11, order 3	+0.0813 / +0.0939	+0.0246 / +0.0441	+0.0631 / +0.0791	+0.0135 ± 0.0015
one-sided, window 11, order 1	+0.1989 / +0.2801	+0.1589 / +0.2485	+0.1652 / +0.2551	+0.0276 ± 0.0017
none	+0.0866 / +0.0510	+0.0468 / +0.0268	+0.0694 / +0.0355	+0.0149 ± 0.0009

rather than something only the analysis code can build. `config/preprocessing.baseball.delay.yaml` is that configuration: depth 3, `hip_shoulder_separation_angle` dropped for stability (Section VII-B), and selection on `persistence_skill_uniform`.

Running the standard fit and evaluate commands on it, held out on the same 29 test clips, changes the operative figures as follows:

- mean derivative R2 goes from -0.0184 to **+0.4411**;
- uniform skill goes from -0.0041 to **+0.4071**;
- pooled skill goes from +0.0014 to +0.2193;
- the candidate sweep is no longer flagged degenerate.

Skill reaches +0.4071, slightly above the analysis script's +0.3881, because the pipeline fits Savitzky-Golay derivatives rather than forward increments directly, so the two routes are not identical and neither is strictly better. The thresholds now separate on validation skill (+0.3076 down to +0.3026), so the selection is meaningful.

The reconstruction metrics on this configuration read -260.7 and must be ignored. A delay state cannot be integrated forward: its delay columns are lagged copies rather than independent degrees of freedom, so a free-running rollout is ill-posed. The evaluator prints a warning whenever it detects delay columns.

This configuration is offered as the recommended direction, not as a drop-in replacement for the operative one. It answers a different question — one-step prediction rather than trajectory reconstruction — and Section V is the reason that is the right question to be asking.

Two further analyses on this model support no biomechanical claim.

Cross-joint coupling and the kinetic chain. Restricting each equation to its own variable's history costs 0.017 uniform skill on the held-out split. Averaged over resplits, coupling is about 1.8 percent of the model's skill. It clears a per-target circular-shift null on four of five held-out splits and is absent on the fifth (`src/analysis/test_coupling_significance.py`). Hips and shoulders are linked through the trunk, so some coupling is expected of any linked body. The proximal-to-distal chain [1], [2] is a claim about direction, so it was tested as the gain from proximal-to-distal terms minus the gain from distal-to-proximal terms (`src/analysis/test_kinetic_chain_asymmetry.py`). The result is +0.0011, 95% CI [-0.0004, +0.0026], positive on 18 of 30 resplits. The asymmetry is not evidenced.

Release velocity and pitch type. Five pre-registered representations were scored:

- a 70-dimensional kinematic summary;
- SINDy coefficients;
- weak-form coefficients;
- the dynamic mode decomposition spectrum [17];
- that spectrum on a delay-embedded state.

The endpoints were `pitch_speed_mph` (RidgeCV, cross-validated R2) and fast versus offspeed class (balanced accuracy). Scoring used 5-fold stratified cross-validation repeated 20 times and a 200-draw label-permutation null [27] (`src/analysis/run_representation_study.`

py). None clears both its trivial baseline and the null. Speed error is 5.32 to 5.37 mph against 5.32 mph for the mean predictor, with p from 0.33 to 0.64. The best class result, balanced accuracy 0.5504 against a 0.4919 baseline, has p = 0.19. With 140 clips only a large effect is detectable, so this null cannot separate absence of signal from low power. Laboratory capture does find kinematic correlates of velocity [13], and Piergiovanni and Ryoo [12] describe the broadcast task as extremely difficult.

VIII. Evaluation lessons

Methodological finding on evaluation. Five intermediate results in this project were false positives. Four were caught by adding the no-model control for the specific task being scored; the fifth by a null the analysis had not originally applied per target:

A sixth near-miss was an aggregation artifact rather than a missing control: pooled skill is positive at every rollout horizon while uniform skill is negative at every horizon (Section V), and reporting only the pooled figure would have produced another spurious positive. The second-order row above was anticipated before the run — position-increment prediction is trivial when velocity is a state variable — so the control was built in and the artifact was caught on first inspection rather than after it had been written up.

The common structure is that all five involved metrics whose absolute values are uninterpretable without a reference point — R2 against a window mean when the target barely moves, and sum-of-squares pooled across columns of very different amplitude. The practice adopted here, and the one this project would recommend to similar work, is that a metric ships with its no-model control and both aggregations before any model is scored against it, not after a result looks surprising. Two established results underwrite it: a score expressed against a trivial reference forecast is interpretable where an absolute one is not [23], and a permutation null computed through the identical pipeline is the test that survives when the same folds have also informed a modelling choice [27], [28].

A selection rule that manufactured a finding. The weak-form model was first recorded as the best available (core R2 -7.89 against -29.56). Its selected model has a single non-bias term, `d(dominant_knee_flexion_angle)/dt = 0.354 * trail_elbow_flexion_angle`, and every other derivative is zero: it is the frozen control plus one term, and it scores worse than the control. The cause is the `sparsity_weighted_r2` rule. When every candidate ties on validation R2, the per-term penalty decides, so the rule minimizes term count; threshold 0.30 won by 1.5e-05. The repaired `accuracy_gated_sparsity` rule keeps the candidates within a tolerance of the best accuracy and takes the sparsest of those, and the fitter flags a sweep `selection_degenerate` when no candidate reaches positive accuracy. The operative first-order configuration selects identically under both rules, so only the weak-form comparison was distorted. Selecting on skill over persistence does not rescue the first-order sweep either: skill spans 0.00088 across the five thresholds, all negative. On the delay state of Section VII the same criterion spans 0.0378 and picks a clear winner. A selection rule can only separate candidates that differ.

One real result was overstated rather than false. The Section VII sparse fit leads an unfitted 2-frame momentum control by 0.26 uniform skill, but a fitted per-joint autoregression by only 0.02 across resplits. The lead survives; its size did not. That is the origin of the local motion rule in Section IV: a fitted model's control is fitted too.

A second overstatement came from the input rather than the control. Persistence was the right reference for the Section VII skill, but the state it was scored on had been smoothed with a centred window, so part of each predicted increment was already in the input. Rebuilt without smoothing, 0.39 becomes 0.05 uniform (Section VII-C). The general form is the Section VI-C oracle again: any preprocessing step that reads future frames must be excluded from a forecasting comparison, or the comparison measures the step.

IX. Limitations and future work

Limitations.

- *Data.* The pool is 140 clips from one broadcast camera view. MediaPipe pose is monocular, largely two-dimensional and noisier than marker-based capture, and camera angle, zoom and occlusion can change feature values independently of the motion.
- *Task.* No configuration shows predictive validity on free-running reconstruction. The Section VII model is a one-step predictor on a delay state and cannot be integrated forward.
- *Smoothing.* The 0.39 skill level depends on a centred filter that reads future frames. Only the unsmoothed figure, 0.05 to 0.09, is achievable as a real-time forecast, and the pipeline's own figures in Section VII-D are all under the centred filter.
- *Interpretation.* Skill over persistence measures increment prediction, not biomechanical validity. Much of it is each joint's own momentum, and the sparse fit's lead over per-joint AR(3), 0.013 to 0.028, is small. The kinetic-chain ordering follows the throwing sequence of Fleisig et al. [2] rather than the data, and the label-study null cannot separate absent signal from low power at 140 clips.
- *Statistics.* Sections V and VI-C report single splits where that is what the pipeline produces. The ensemble and second-order rows of Table I have no intervals, and the H = 240 horizon result rests on 13 clips.
- *Model selection.* `sparsity_weighted_r2` degenerates when candidates tie (Section VIII), so selected sparsity levels are not evidence of a parsimonious true model.

Future research directions.

TABLE VII
INTERMEDIATE RESULTS THAT DID NOT SURVIVE THE CONTROL OR NULL FOR THEIR TASK.

apparent result	control that overturned it	outcome
weak-form core R2 -7.89, "3.7x better than baseline"	frozen zero-dynamics model, -5.57	dissolved; the model had one non-bias term
delay-embedding derivative R2 +0.69	unfitted backward difference, +0.35	roughly half was numerical differentiation
teacher-forced one-step core R2 +0.76	persistence, +0.85	dissolved
second-order short-horizon skill +0.89	its own derivative reference, +0.89	dissolved; velocity is an input column
proximal-to-distal asymmetry +0.0011, four of five splits	the same null applied per target, over 30 resplits	dissolved; interval [-0.0004, +0.0026], positive on 18 of 30
clearing a shift null		

1. *Forecasting as the target task.* The open question is whether structure beyond per-joint momentum survives on causally smoothed or unsmoothed pose, and over horizons longer than one frame. Longer clips would also test the crossover hinted at H = 240.
2. *Better pose.* Pose-quality screening, multi-view or 3D pose would test whether the remaining lead grows as measurement noise falls. Batting needs a different pose model: the batter is detected in only 78 percent of clips, and upscaling the crop does not fix it.
3. *External validation.* A larger labelled pool would let the release-velocity and pitch-type study separate absent signal from low power. A biomechanical claim would have to start there.

Engineering follow-ups, such as re-scoring the alignment track and adding causal smoothing to the pipeline, are tracked in the repository.

X. Conclusion

Sparse dynamical models fitted to broadcast-derived pitching pose do not reconstruct held-out motion. Every configuration tried — first-order, second-order, ensemble and weak-form — is beaten by a predictor that holds the initial pose fixed, and accuracy falls as model complexity rises. Direct tests make phase, clip heterogeneity, the pose noise floor and the sparse estimator unlikely explanations, and point to the state representation: joint positions alone do not determine the next increment.

Adding two frames of history supports short-horizon prediction, and the project's own sparse estimator recovers it with 30 terms, matching a 240-coefficient dense model. Most of the measured level, however, comes from the centred smoothing filter (0.39 skill with it, 0.05 to 0.09 without), and most of the rest is each joint's own momentum. What holds under every smoother is the sparse fit's small lead over fitted per-joint momentum, 0.01 to 0.03, positive on all 40 resplits. Joint coupling is weak, the directional kinetic-chain asymmetry is not evidenced, and release velocity and pitch type are not predictable from any pre-registered representation.

The contributions are a reproducible pipeline from broadcast video to sparse equations and a control-paired evaluation protocol. The protocol is the transferable part. Five intermediate results here looked positive and did not survive the control or null for their task, and two real results were overstated until a fitted control and an unsmoothed rebuild were added. A score whose absolute value is uninterpretable should ship with its control and both aggregations before any model is judged by it.

These results are not evidence of governing laws of pitching mechanics. The discovered coefficients predict increments but have not been validated against any external quantity, and the 0.39 figure is not a real-time forecast.

XI. Reproducibility

Code, configurations and the train/test split files are public.[1] `docs/reproducing.md` in the repository gives the command that regenerates each table and figure, and `src/analysis/make_paper_figures.py` renders the figures. Raw video, pose and generated arrays are not redistributed; they are rebuilt from MLB-YouTube [12] and UTD-MHAD [18]. The operative configuration must be passed explicitly, because the refresh script's default configuration does not reproduce Section V.

XII. References

[1] G. S. Fleisig, J. R. Andrews, C. J. Dillman, and R. F. Escamilla, "Kinetics of baseball pitching with implications about injury mechanisms," *Amer. J. Sports Med.*, vol. 23, no. 2, pp. 233–239, Mar./Apr. 1995, doi: 10.1177/036354659502300218.

[2] G. S. Fleisig, S. W. Barrentine, R. F. Escamilla, and J. R. Andrews, "Biomechanics of overhand throwing with implications for injuries," *Sports Med.*, vol. 21, no. 6, pp. 421–437, Jun. 1996, doi: 10.2165/00007256-199621060-00004.

[3] R. M. Kanko, E. K. Laende, E. M. Davis, W. S. Selbie, and K. J. Deluzio, "Concurrent assessment of gait kinematics using marker-based and markerless motion capture," *J. Biomech.*, vol. 127, art. no. 110665, Oct. 2021, doi: 10.1016/j.jbiomech.2021.110665.

[4] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proc. Nat. Acad. Sci.*, vol. 113, no. 15, pp. 3932–3937, Apr. 2016, doi: 10.1073/pnas.1517384113.

[5] D. A. Messenger and D. M. Bortz, "Weak SINDy: Galerkin-based data-driven model selection," *Multiscale*

[1] https://github.com/ka-acy/sindy-pitching

Model. Simul., vol. 19, no. 3, pp. 1474–1497, 2021, doi: 10.1137/20M1343166.

[6] D. A. Messenger and D. M. Bortz, "Weak SINDy for partial differential equations," *J. Comput. Phys.*, vol. 443, art. no. 110525, 2021, doi: 10.1016/j.jcp.2021.110525.

[7] U. Fasel, J. N. Kutz, B. W. Brunton, and S. L. Brunton, "Ensemble-SINDy: Robust sparse model discovery in the low-data, high-noise limit, with active learning and control," *Proc. Roy. Soc. A*, vol. 478, no. 2260, art. no. 20210904, 2022, doi: 10.1098/rspa.2021.0904.

[8] K. Champion, P. Zheng, A. Y. Aravkin, S. L. Brunton, and J. N. Kutz, "A unified sparse optimization framework to learn parsimonious physics-informed models from data," *IEEE Access*, vol. 8, pp. 169259–169271, 2020, doi: 10.1109/ACCESS.2020.3023625.

[9] B. M. de Silva, K. Champion, M. Quade, J.-C. Loiseau, J. N. Kutz, and S. L. Brunton, "PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data," *J. Open Source Softw.*, vol. 5, no. 49, art. no. 2104, 2020, doi: 10.21105/joss.02104.

[10] A. A. Kaptanoglu et al., "PySINDy: A comprehensive Python package for robust sparse system identification," *J. Open Source Softw.*, vol. 7, no. 69, art. no. 3994, 2022, doi: 10.21105/joss.03994.

[11] M. C. Rosenberg, J. L. Proctor, and K. M. Steele, "Quantifying changes in individual-specific template-based representations of center-of-mass dynamics during walking with ankle exoskeletons using Hybrid-SINDy," *Sci. Rep.*, vol. 14, art. no. 1031, 2024, doi: 10.1038/s41598-023-50999-0.

[12] A. J. Piergiovanni and M. S. Ryoo, "Fine-grained activity recognition in baseball videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR) Workshops*, 2018. [Online]. Available: https://arxiv.org/abs/1804.03247

[13] T. Matsuo, G. S. Fleisig, R. F. Escamilla, S. W. Barrentine, and J. R. Andrews, "Comparison of kinematic and temporal parameters between different pitch velocity groups," *J. Appl. Biomech.*, vol. 17, no. 1, pp. 1–13, 2001.

[14] A. L. Aguinaldo, J. Buttermore, and H. Chambers, "Effects of upper trunk rotation on shoulder joint torque among baseball pitchers of various levels," *J. Appl. Biomech.*, vol. 23, no. 1, pp. 42–51, Feb. 2007, doi: 10.1123/jab.23.1.42.

[15] F. Takens, "Detecting strange attractors in turbulence," in *Dynamical Systems and Turbulence, Warwick 1980* (Lecture Notes in Mathematics, vol. 898), D. A. Rand and L.-S. Young, Eds. Berlin, Germany: Springer, 1981, pp. 366–381, doi: 10.1007/BFb0091924.

[16] S. L. Brunton, B. W. Brunton, J. L. Proctor, E. Kaiser, and J. N. Kutz, "Chaos as an intermittently forced linear system," *Nature Commun.*, vol. 8, art. no. 19, 2017, doi: 10.1038/s41467-017-00030-8.

[17] P. J. Schmid, "Dynamic mode decomposition of numerical and experimental data," *J. Fluid Mech.*, vol. 656, pp. 5–28, Jul. 2010, doi: 10.1017/S0022112010001217.

[18] C. Chen, R. Jafari, and N. Kehtarnavaz, "UTD-MHAD: A multimodal dataset for human action recognition utilizing a depth camera and a wearable inertial sensor," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2015, pp. 168–172, doi: 10.1109/ICIP.2015.7350781.

[19] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019. [Online]. Available: https://arxiv.org/abs/1906.08172

[20] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020. [Online]. Available: https://arxiv.org/abs/2006.10204

[21] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," *Anal. Chem.*, vol. 36, no. 8, pp. 1627–1639, 1964, doi: 10.1021/ac60214a047.

[22] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *Int. J. Forecasting*, vol. 22, no. 4, pp. 679–688, 2006, doi: 10.1016/j.ijforecast.2006.03.001.

[23] A. H. Murphy, "Skill scores based on the mean square error and their relationships to the correlation coefficient," *Mon. Weather Rev.*, vol. 116, no. 12, pp. 2417–2424, Dec. 1988, doi: 10.1175/1520-0493(1988)116<2417:SSBOTM>2.0.CO;2.

[24] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.

[25] S. Ross and J. A. Bagnell, "Efficient reductions for imitation learning," in *Proc. 13th Int. Conf. Artif. Intell. Statist. (AISTATS)*, PMLR, 2010, pp. 661–668.

[26] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Inf. Sci.*, vol. 191, pp. 192–213, 2012, doi: 10.1016/j.ins.2011.12.028.

[27] M. Ojala and G. C. Garriga, "Permutation tests for studying classifier performance," *J. Mach. Learn. Res.*, vol. 11, pp. 1833–1863, 2010.

[28] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, art. no. 91, 2006, doi: 10.1186/1471-2105-7-91.

[29] K. Sun, B. Xiao, D. Liu, and J. Wang, "Deep high-resolution representation learning for human pose estimation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 5693–5703.

[30] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten simple rules for reproducible computational research,"

PLoS Comput. Biol., vol. 9, no. 10, art. no. e1003285, 2013, doi: 10.1371/journal.pcbi.1003285.

[31] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[32] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[33] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.